

LECTURE 12

Scheduling Algorithms: Multilevel Queues and Real-World Systems

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Multilevel queue scheduling: partitioning the ready queue by process type
- Multilevel feedback queue scheduling: letting processes move between queues based on observed behavior, and why that approximates SJF without knowing burst lengths in advance
- How thread scheduling connects back to Lecture 9's multithreading models via contention scope
- Linux's Completely Fair Scheduler (CFS) and a brief look at Windows' scheduler

Multilevel Queue Scheduling

A multilevel queue scheduler splits the single ready queue into several separate queues, grouped by process type — commonly a queue for interactive, foreground processes and a different queue for CPU-bound background (batch) processes.

Each queue can run its own scheduling algorithm internally (foreground might use Round Robin for responsiveness; background might use FCFS, since nobody is watching it in real time), and a separate policy decides how the CPU is shared between the queues — usually fixed priority (always service foreground first) or time slicing (e.g. 80% of CPU to foreground, 20% to background).

A Multilevel Queue: Foreground vs. Background

Foreground Queue

Interactive processes, scheduled with Round Robin. Serviced first — or given the larger time slice.

Background Queue

CPU-bound batch processes, scheduled with FCFS. Runs only when foreground is empty — or given the smaller time slice.

A structural weakness

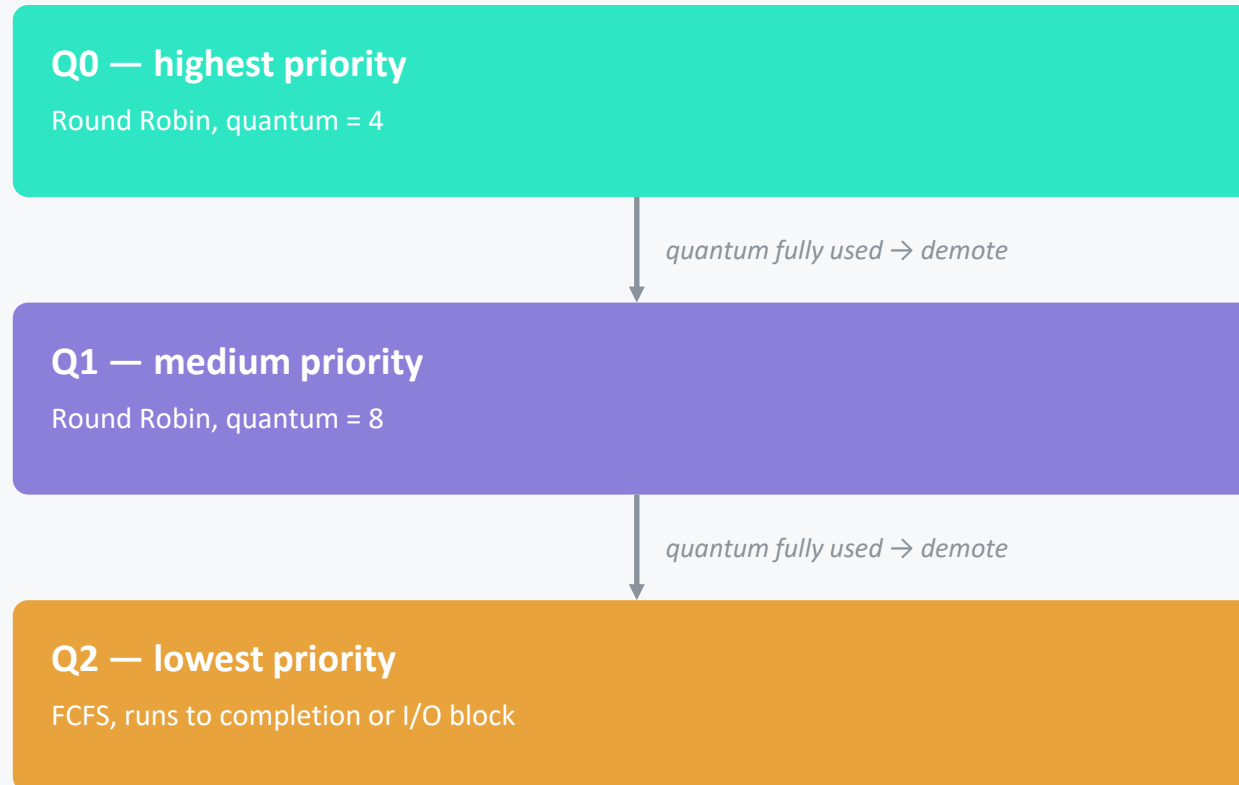
Multilevel queue scheduling is simple and lets each class of process be scheduled appropriately, but a process assigned to a queue stays in that queue for its entire lifetime — a process classified as "background" at creation time is stuck there even if its actual behavior turns out to look nothing like a batch job.

Multilevel Feedback Queue (MLFQ)

A multilevel feedback queue fixes that weakness by letting processes move between queues based on how they actually behave, observed at run time rather than assumed at creation time.

- **Uses its entire time quantum without blocking** — behaving like a CPU-bound process. It gets demoted to the next, lower-priority queue, which has a longer quantum (fewer context switches needed).
- **Blocks on I/O before its quantum expires** — behaving like an I/O-bound, interactive process. It stays in its current queue (or moves back up), so next time it's ready, it's serviced quickly.

Three Levels: Demotion on a Full Quantum



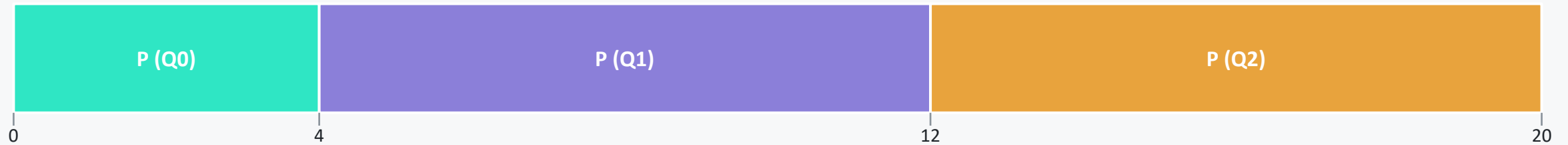
Worked Example: One Process Sinking Through the Queues

A three-level MLFQ (Q0 quantum 4, Q1 quantum 8, Q2 FCFS) receives one CPU-bound process P, burst 20, every queue otherwise empty. P enters at the top, Q0.

- **In Q0**, P runs its full quantum, 4 units ($t=0 \rightarrow 4$), without blocking — remaining $20-4=16$. Quantum fully used $\rightarrow$ demoted to Q1.
- **In Q1**, P runs its full quantum, 8 units ($t=4 \rightarrow 12$) — remaining $16-8=8$. Quantum fully used again $\rightarrow$ demoted to Q2.
- **In Q2** (FCFS — no quantum to expire), P simply runs to completion: its remaining 8 units, $t=12 \rightarrow 20$.

Gantt Chart: One CPU-Bound Process, Sinking from Q0 to Q2

P's single 20-unit burst, demoted at each full quantum



Had P instead been a short, interactive process with a burst of only 3, it would have finished entirely within its very first Q0 turn — never demoted at all, completing at t=3 without ever waiting behind P's remaining 17 units of work.

MLFQ Approximates SJF — Without Knowing the Future

MLFQ never needs to know a process's burst length in advance the way SJF does — it simply watches whether a process keeps using its entire quantum, and demotes it when it does.

A genuinely short process is, by definition, unlikely to ever use a full quantum before finishing or blocking, so it naturally stays near the top, getting serviced quickly — while a genuinely long, CPU-bound process naturally sinks toward the bottom, where longer quanta mean fewer wasted context switches. This is why MLFQ is considered the most general scheduling approach, and the one most real-world systems are built around in some form.

MLFQ is defined by its rules, not by one fixed configuration

The number of queues, each queue's quantum, and the exact promotion/demotion rules are all parameters a specific OS chooses — there is no single canonical MLFQ. What makes an algorithm "an MLFQ" is the shape of the idea: multiple priority levels, movement driven by observed behavior rather than a fixed, permanent classification.

Thread Scheduling and Contention Scope

Lecture 9 distinguished user threads, managed by a thread library, from kernel threads, scheduled directly by the OS. That distinction decides which scheduler actually decides when a thread runs — its contention scope.

Process-Contention Scope (PCS)

Under many-to-one or many-to-many: the thread library decides which of a process's own user threads runs next. Threads only compete against other threads in the same process.

System-Contention Scope (SCS)

Under one-to-one: the kernel schedules every thread directly, so a thread competes against every other thread on the system, regardless of process.

Setting Contention Scope in Pthreads

The Pthreads API exposes this choice directly through `pthread_attr_setscope()`, which takes either `PTHREAD_SCOPE_PROCESS` (PCS) or `PTHREAD_SCOPE_SYSTEM` (SCS).

Linux's one-to-one model makes SCS the only scope that actually matches its underlying kernel threading — the PCS/SCS distinction matters far more on systems that genuinely support many-to-many scheduling.

Linux's Completely Fair Scheduler (CFS)

Linux's default scheduler since kernel 2.6.23 abandons fixed time quanta and priority numbers in favor of one running number per task: virtual runtime (vruntime), tracking how much CPU time a task has already received, weighted by its priority.

CFS's entire scheduling rule

Always run the ready task with the smallest vruntime. A task that has run less than others is, by definition, "owed" more CPU time to stay fair, so it's the one that runs next; the moment it runs, its vruntime climbs, and eventually some other task becomes the one with the smallest vruntime instead.

Tracing CFS: Three Equal-Priority Tasks

Each task gets a slice of 2 time units whenever chosen; all vruntimes start at 0.

Decision	Vruntimes Before (A, B, C)	Task Chosen	Vruntimes After
1	0, 0, 0	A (tie, chosen first)	2, 0, 0
2	2, 0, 0	B	2, 2, 0
3	2, 2, 0	C	2, 2, 2
4	2, 2, 2	A (tie again)	4, 2, 2

With equal priority, CFS's min-vruntime rule converges to exactly the same round-robin rotation Lecture 11 already covered — each task runs, falls behind no one, and gets picked again once everyone else catches up.

CFS with Unequal Priorities

The real payoff shows up once priorities (Linux calls them nice values) differ: a higher-priority task's vruntime is scaled to grow more slowly per unit of actual CPU time it receives, so it reaches the "smallest vruntime" threshold again sooner and gets scheduled more often.

All without CFS ever needing a separate queue per priority level, a fixed quantum, or any of MLFQ's explicit demotion rules. This single running number is what lets CFS approximate an ideal, perfectly fair division of the CPU proportional to priority, across any number of tasks.

Real-World Scheduling: Windows

Windows uses a priority-based, preemptive scheduler with 32 priority levels, organized into priority classes a process belongs to and relative priorities within that class for each of its threads. Threads at higher priority levels preempt lower ones.

Windows dynamically boosts the priority of a thread that just finished waiting on I/O (rewarding interactive, I/O-bound behavior), while decaying that boost back down over time as the thread continues to run.

The Same Idea, Implemented Differently

Windows' I/O boost-and-decay is exactly MLFQ's "stay near the top if you block before your quantum expires" rule, and the decay mirrors MLFQ's demotion rule — the same underlying idea, implemented through dynamic priority adjustment rather than literally moving a thread between separate queue data structures.

Structurally, Windows behaves like a multilevel-feedback-queue system, even though it never explicitly names separate queues the way the MLFQ model does.

Key Takeaways

- Multilevel queue scheduling partitions the ready queue by process type, each with its own algorithm, but permanently fixes which queue a process belongs to
- Multilevel feedback queue scheduling lets processes move between queues based on observed behavior — a full quantum demotes toward longer-quantum, lower-priority queues; blocking early keeps a process near the top — approximating SJF without ever needing to know burst lengths in advance
- Contention scope connects back to Lecture 9's multithreading models: process-contention scope (PCS) under many-to-one/many-to-many, system-contention scope (SCS) under one-to-one, selectable in Pthreads via `pthread_attr_setscope()`
- Linux's CFS replaces quanta and priority queues with one number, `vruntime`, always running the ready task with the smallest value — approximating ideal fair sharing directly
- Windows runs a 32-level, priority-based preemptive scheduler that boosts a thread's priority after I/O waits and decays it with continued CPU use — the same underlying idea as MLFQ's demotion rule, applied through dynamic priority rather than separate queues